

Stephanie Rawls

MTRE 6100

Advanced Robotic Programming

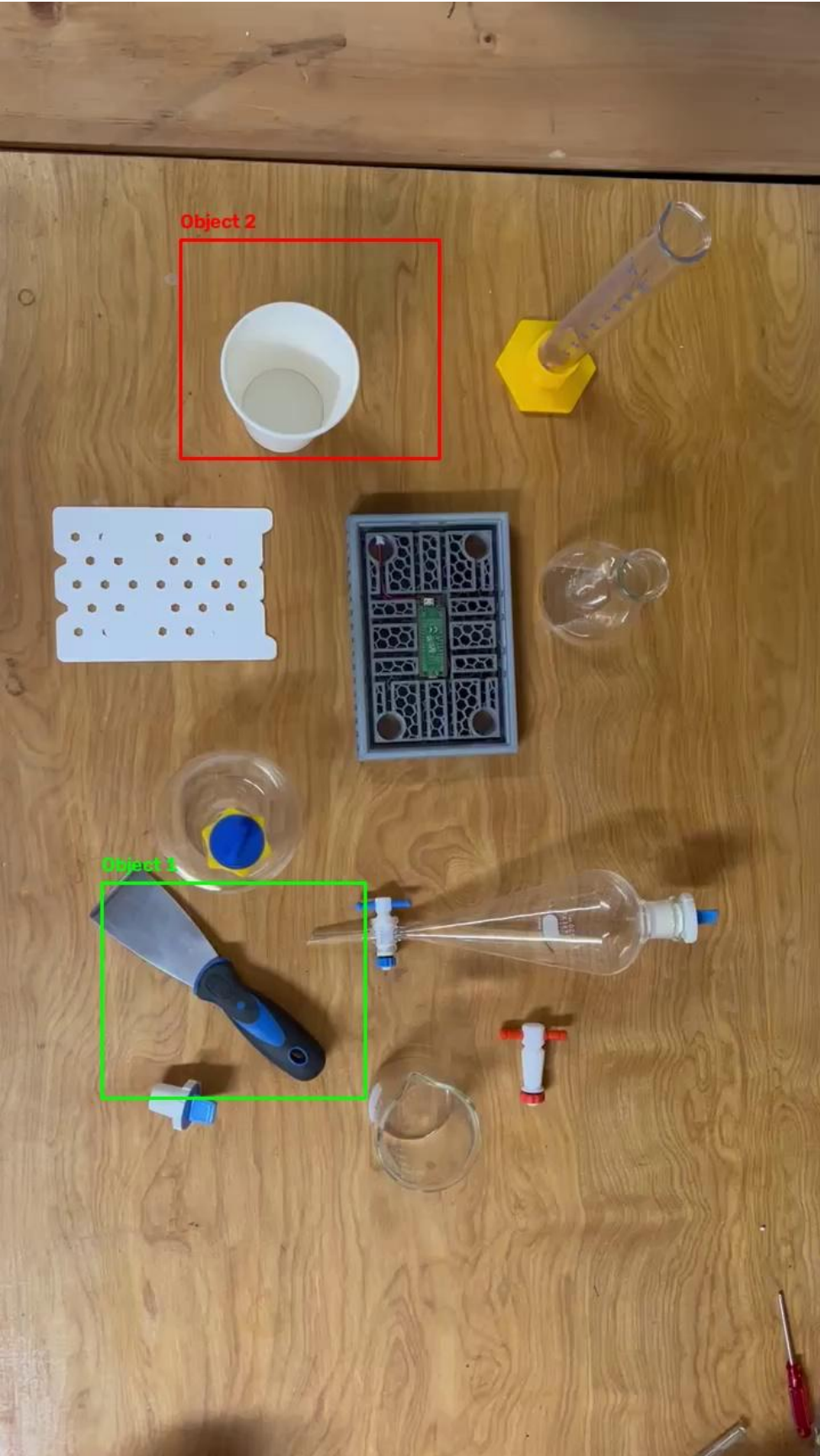
23 September 2026

HW 3 Assignment

Template Images Bench Scraper and Cup





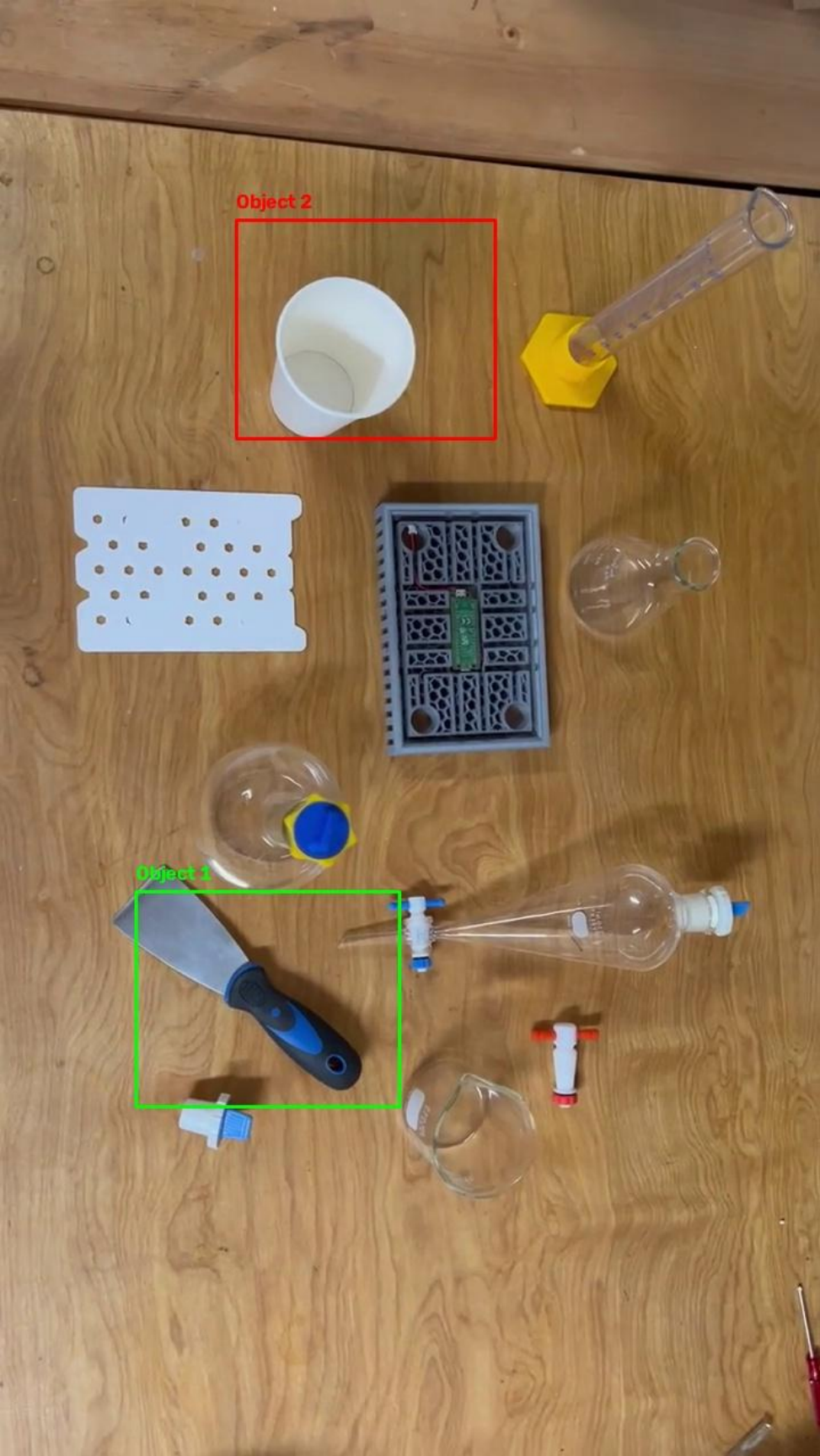


Object 2



Object 1





Object 2



Object 1

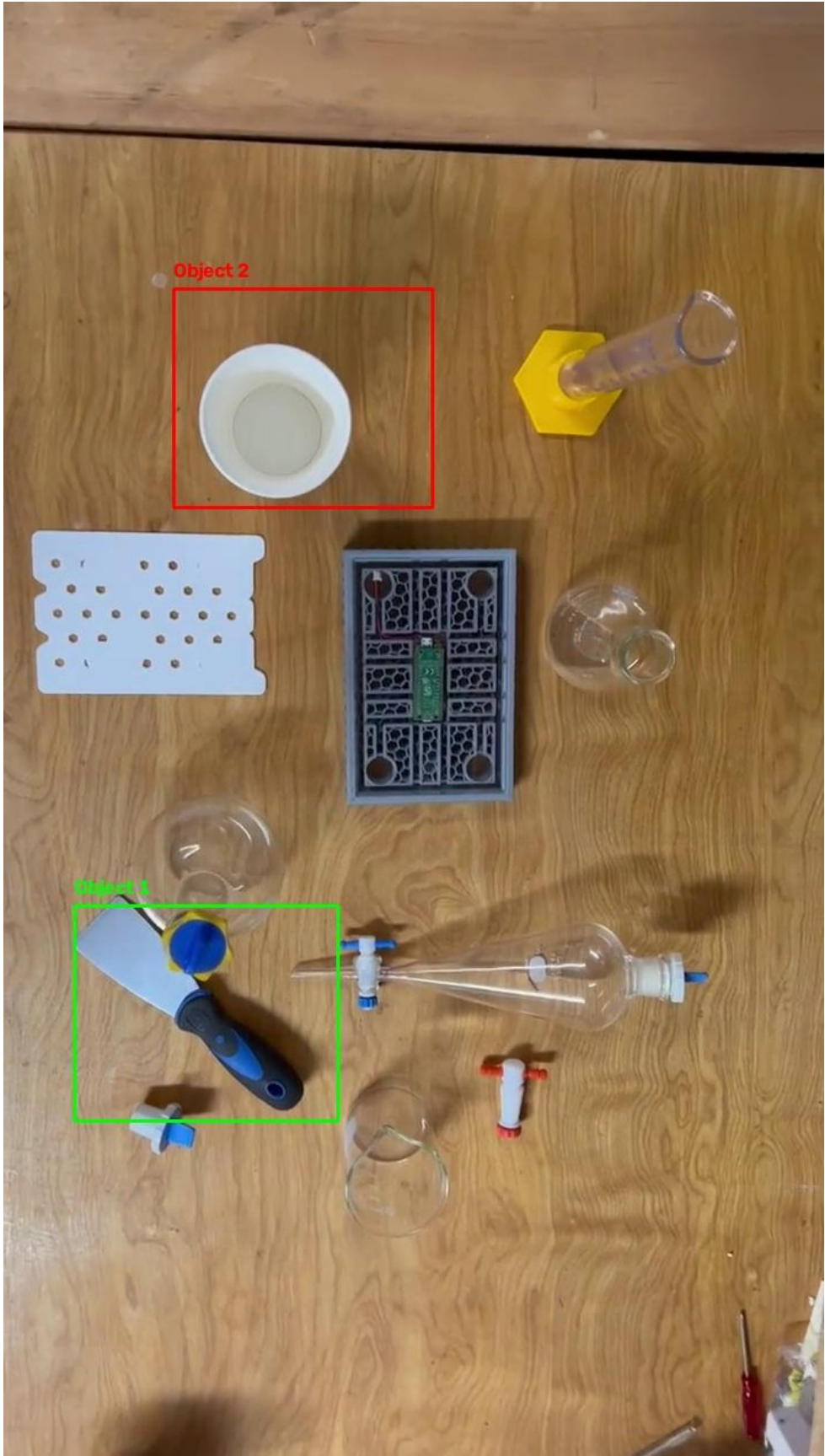


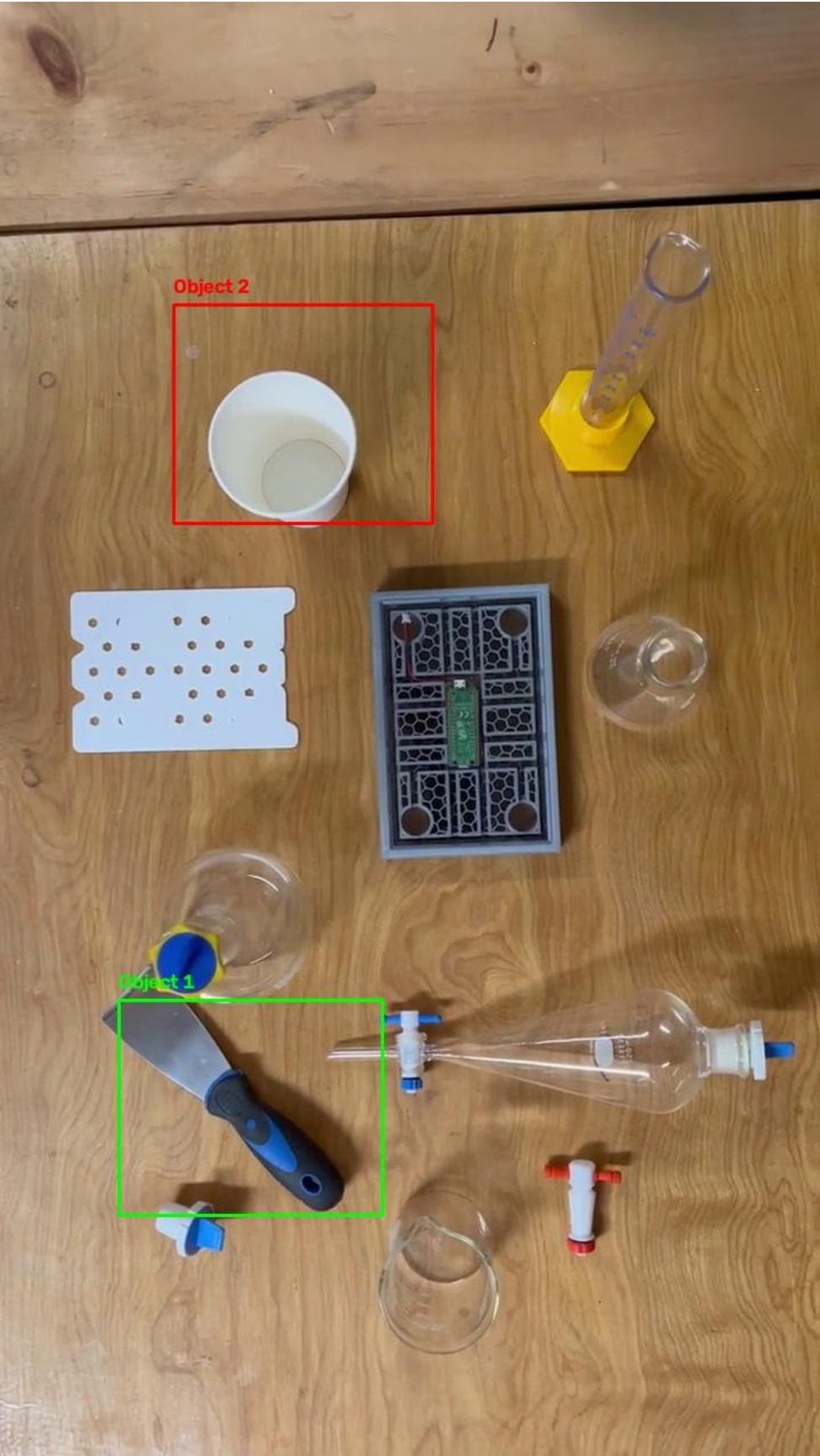
Object 2



Object 1







Python Code Copy

```
import cv2 as cv
```

```
import sys
```

```
threshold = 0.6
```

```
def imageimport(baseimage, image1name, image2name):
```

```
    # Import the images that are needed
```

```
    img1 = cv.imread(cv.samples.findFile(image1name), cv.IMREAD_GRAYSCALE)
```

```
    img2 = cv.imread(cv.samples.findFile(image2name), cv.IMREAD_GRAYSCALE)
```

```
    baseimg = cv.imread(cv.samples.findFile(baseimage), cv.IMREAD_GRAYSCALE)
```

```
    # display your images in a window
```

```
    if img1 is None or img2 is None or baseimg is None:
```

```
        sys.exit('Images Not Found')
```

```
    else:
```

```
        cv.imshow("Image 1", img1)
```

```
        cv.imshow("Image 2", img2)
```

```
        cv.imshow("Base Image", baseimg)
```

```
        w1, h1 = img1.shape[::-1]
```

```
        w2, h2 = img2.shape[::-1]
```

```
        w, h = baseimg.shape[::-1]
```

```
    k = cv.waitKey(0)
```

```
    # This function typically provides the time in milliseconds to display the window.
```

```
# 0 means wait forever

if k == ord('s'):

    cv.imwrite(image1name, img1)

    cv.imwrite(image2name, img2)

    cv.imwrite(baseimage, baseimg)


cv.destroyAllWindows()


# hand everything back so templatematching can use it

return img1, w1, h1, img2, w2, h2


def templatematching(videoname, img1, w1, h1, img2, w2, h2):

    # Loading in the video file into the environment

    video = cv.VideoCapture(videoname)


    # Define the codec and create VideoWriter object to save the annotated video

    fourcc = cv.VideoWriter_fourcc(*'mp4v')

    frame_width = int(video.get(cv.CAP_PROP_FRAME_WIDTH))

    frame_height = int(video.get(cv.CAP_PROP_FRAME_HEIGHT))

    frame_rate = video.get(cv.CAP_PROP_FPS)

    out = cv.VideoWriter('detected_video.mp4', fourcc, frame_rate, (frame_width,
frame_height))


    # keep track of frames where both objects were found, for the screenshots

    detected_frames = []

    frame_count = 0
```

```
# capture each frame of the video and store it in a variable
while video.isOpened():

    success, frame = video.read()

    if not success:

        print("Can't receive frame (stream end?). Exiting ...")
        break

# Graying the singular frames allows for more contrast and better detection
video_frames = cv.cvtColor(frame, cv.COLOR_BGR2GRAY)

objects_found = 0

# Bench scraper detection

# Actual command for template matching. This will find the image in the video
# frame and return a value that is used to determine if the image is present or not.
matching_results_1 = cv.matchTemplate(video_frames, img1,
cv.TM_CCOEFF_NORMED)

# Tells us where the object is located in the video frame, based on coordinates.
# Highest correlation value is the best match.
min_val_1, max_val_1, min_loc_1, max_loc_1 = cv.minMaxLoc(matching_results_1)

# Adding in certainty...we need our value to be greater than a threshold
if max_val_1 >= threshold:

    top_left_1 = max_loc_1

    bottom_right_1 = (top_left_1[0] + w1, top_left_1[1] + h1)
```

```

cv.rectangle(frame, top_left_1, bottom_right_1, (0, 255, 0), 2)

# Neat little trick :)

cv.putText(frame, 'Bench Scraper', (top_left_1[0], top_left_1[1] - 10),
           cv.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)

objects_found = objects_found + 1


# Cup

matching_results_2 = cv.matchTemplate(video_frames, img2,
cv.TM_CCOEFF_NORMED)

min_val_2, max_val_2, min_loc_2, max_loc_2 = cv.minMaxLoc(matching_results_2)

if max_val_2 >= threshold:

    top_left_2 = max_loc_2

    bottom_right_2 = (top_left_2[0] + w2, top_left_2[1] + h2)

    cv.rectangle(frame, top_left_2, bottom_right_2, (0, 0, 255), 2)

    cv.putText(frame, 'Object 2', (top_left_2[0], top_left_2[1] - 10),
               cv.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)

    objects_found = objects_found + 1


# write this frame to the output video and show it live

out.write(frame)

cv.imshow('Detected Frame', frame)


# if both objects were found in this frame, keep a copy for screenshots later

if objects_found == 2:

    detected_frames.append(frame.copy())

```

```
frame_count = frame_count + 1
```

```
# close video display with q
```

```
if cv.waitKey(1) == ord('q'):
```

```
    break
```

```
video.release()
```

```
out.release()
```

```
cv.destroyAllWindows()
```

```
print('Processed', frame_count, 'frames')
```

```
print('Frames with both objects detected:', len(detected_frames))
```

```
# save 7 screenshots, evenly spaced across all the successful detections
```

```
num_screenshots = 7
```

```
if len(detected_frames) > 0:
```

```
    if len(detected_frames) < num_screenshots:
```

```
        num_screenshots = len(detected_frames)
```

```
    step = len(detected_frames) / num_screenshots
```

```
    for i in range(num_screenshots):
```

```
        index = int(i * step)
```

```
        cv.imwrite('detection_' + str(i + 1) + '.jpg', detected_frames[index])
```

```
def main():
```

```
    img1, w1, h1, img2, w2, h2 = imageimport('baseimage.png', 'scraper.png', 'cup.png')
```

```
templatematching('video2.mp4', img1, w1, h1, img2, w2, h2)
```

```
if __name__ == "__main__":
```

```
    main()
```